AlphaTerra Data Pipeline Architecture Overview

Assignment: Building a Real Estate Data Intelligence Engine (Hyderabad). Covers deliverable (a) architecture and the pipeline part of deliverable (d).

Overview

The pipeline turns raw property information into trusted data and then into useful decisions. It has 10 stops, grouped into 4 stages. Three concerns run across every stage: price history, monitoring and governance.

Architecture diagram (text version)

Sources (portals, RERA, maps, app events) → **Collect** (keep raw copy) → **Clean** (check and match) → **Store** (canonical tables) → **Use** (scores, charts)

Across every stage: history, monitoring, governance.

The 4 stages and 10 stops

Stage	Stops it covers	What happens
Collect	1 Source acquisition, 2 Raw storage	Download the data and keep an untouched copy
Clean	3 Validation, 4 Entity resolution, 5 Standardisation	Check for errors, match duplicates, make units and spellings consistent
Store	6 Canonical models, 7 Analytical warehouse	Save it in the clean tables from the data model
Use	8 Feature generation, 9 Intelligence services, 10 Dashboards and APIs	Make useful numbers, scores and clusters, then show them to people

What runs across every stage

- **Price history:** nothing is overwritten. A price change adds a new row with a timestamp.
- **Monitoring:** alerts for failed jobs, stale data and sudden changes in the numbers.
- **Governance:** records where each record came from, when, and what was done to it, plus access control and retention rules.

How the stages map to the proof of concept

Pipeline stop	Proof-of-concept equivalent
Source acquisition	make_data.py
Raw storage	data/properties.csv
Validation	validate.py
Standardisation	partly in validate.py
Canonical models	the SQL schema file
Feature generation	price per sq ft column
Intelligence services	cluster2.py
Dashboards	charts.py

Entity resolution and the analytical warehouse are not built in the proof of concept. They are covered in the design only.

Technology choices and why

- **Python** scripts for ingestion and cleaning.
- **PostgreSQL** as both the database and the warehouse.
- **Plain files or cloud storage** for raw copies.
- **A simple scheduler** (cron, or a small tool later) to run jobs.

Assumption: one city's listings is tens of thousands to a few hundred thousand rows. PostgreSQL handles that easily and cheaply, and one engineer can run it. Tools built for billions of rows, such as Spark or Kafka, would add cost and complexity with no benefit at

this scale. They can be added if the scale changes.

Deliverable status

- (a) Architecture document: this document plus the diagram.
- (c) Data model: SQL schema done; the ERD picture is still to do.
- (d) Pipeline specification: the stops and capability answers above.
- (f) Working proof of concept: done (synthetic data, validation, clustering, charts).

Next step

Step 4: the three intelligence products, one at a time.

- Product A, Property Intelligence Score: is this property a good deal?
- Product B, Locality Intelligence: which areas are getting better or worse?
- Product C, Risk Intelligence: which listings look suspicious?

Report map

This document is the single report for the AlphaTerra assignment. The sections above cover the architecture and the pipeline overview. The sections below cover everything else. Companion files in the project folder: AlphaTerra_Data_Source_Register.xlsx, alphaterra_schema.sql, make_data.py, validate.py, cluster2.py, charts.py, run_all.py, requirements.txt and README.md.

Deliverable	Where to find it
(a) Architecture document	Sections above (stage table and text diagram). Add the diagram image saved as docs/pipeline_diagram.png
(b) Data source register	Data source register below. Full version in AlphaTerra_Data_Source_Register.xlsx
(c) Data model	Canonical data model below. Full SQL in alphaterra_schema.sql
(d) Pipeline specification	Sections above plus Pipeline specification in detail

Deliverable	Where to find it
(e) Intelligence product specification	Intelligence products (A, B, C) and Investor behaviour intelligence
(f) Working proof of concept	Proof of concept and Appendix A
(g) Business case	Prioritisation and Business case
(h) SQL, code, dictionary, assumptions, instructions	Appendices, Assumptions and limitations

Executive summary

AlphaTerra needs an intelligence engine that answers five questions reliably: which properties are attractive, what a property is defensibly worth, which localities are strengthening or weakening, which listings or records carry risk, and which insights improve conversion and liquidity.

This report proposes a small, proportionate system: Python jobs, PostgreSQL, and an append-only raw layer. It does not use distributed infrastructure, because the expected scale (one city, tens of thousands to a few hundred thousand listings) does not need it.

The recommendation is to build the Property and Listing Risk Intelligence capability first, as a data trust layer. It uses data that can be obtained lawfully now, it produces the cleaned and matched records that the score and locality products depend on, and it gives a measurable result within about four weeks. The recommendation changes if funnel data already exists and shows a large drop-off (see Prioritisation).

The proof of concept uses synthetic data for Madhapur, Khajaguda, Nanakramguda and Narsingi. It demonstrates ingestion, validation, standardisation and clustering into low budget, mid range and high segments. Its results show the method, not real market facts.

Data source register

Every source is classified as verified, self-reported, inferred, estimated or unavailable. No source is assumed to be accessible, complete, current or legally reusable.

Fields collected for each listing: property identifier, project name, developer, locality, coordinates, property type, carpet area, built-up area, land area where relevant, configuration, age, floor, amenities, asking price, price per square foot, listing status,

furnishing status, source name, reliability class, URL, and the date the record was checked.

#	Dataset	Source	Frequency	Class	Access and legal note
1	Sale listings	Listing portals (partner feeds), own listings	Daily to weekly	Self-reported	Portal terms usually forbid scraping. Use feeds or permitted samples
2	Developer project pages	Developer websites	Monthly	Self-reported	Public. Cite, do not copy
3	Registered sale deeds	Telangana Registration and Stamps Dept	Monthly	Verified (declared value)	Per-property lookups lawful. Bulk reuse needs permission
4	Guideline values	Telangana Registration Dept	Occasional	Verified (a floor, not a market price)	Public
5	Market reports	Knight Frank, JLL, Anarock, PropEquity	Quarterly	Estimated	Cite, do not reproduce
6	Base map and points of interest	OpenStreetMap	Monthly	Self-reported (crowd-sourced)	ODbL licence: attribution and share-alike
7	Metro and transit	Hyderabad Metro Rail, HMDA, TSRTC	Quarterly	Verified	Public
8	Employment centres	Manual list (HITEC City, Financial District, Gachibowli)	Yearly review	Inferred	Manual compilation
9	Flood and terrain	Bhuvan, NDMA, GHMC lake maps, SRTM elevation	Yearly review	Estimated	Patchy coverage. Mark unavailable where absent
10	RERA registrations	Telangana RERA portal	Weekly	Verified	Public disclosure. Check terms for bulk collection
11	Building permissions	HMDA, GHMC, DTCP	On request or monthly	Verified where accessible	Mostly per-property lookup
12	Litigation and encumbrance	eCourts, IGRS encumbrance search	On demand	Verified (hard to link)	Lawful for due diligence. Route to human and legal review

#	Dataset	Source	Frequency	Class	Access and legal note
13	Developer delivery history	RERA history, developer sites, news	Monthly	Verified (RERA) or self-reported	Public
14	Corporate identity	MCA21	Quarterly	Verified	Basic data public. Detail may be paid
15	Asking rents	Rental listings	Weekly	Self-reported	Same portal terms as sale listings
16	Registered rent agreements	Registration dept	Rare	Unavailable	Not bulk accessible. Use asking rent with a haircut (Estimated)
17	Rental yield	Computed	Each refresh	Estimated	Internal
18	Application events	AlphaTerra app and backend	Real time or hourly	Verified (own system)	DPDP Act 2023: consent, purpose limitation, minimisation
19	Payments and refunds	Payment gateway and ledger	Real time	Verified	Financial data, restricted access
20	Support interactions	CRM or helpdesk	Daily	Self-reported	Contains personal data. Mask it

Asking price versus transaction price

- Asking price (listings) is self-reported, negotiable and can be stale. It is stored as a price observation with type asking.
- Transaction price (registered deeds) is verified, but the declared consideration may understate the true price. It is stored with type registered.
- Where both exist for a locality, an asking-to-registered ratio is computed and reported as an estimated discount, never as a fact.
- Where no transaction data is available, it is marked unavailable. Time on market and price-cut frequency are used as liquidity proxies.

Gaps and lawful alternatives

Gap	Why it exists	Lawful alternative
True transaction prices	Under-declaration, limited bulk access	Guideline values as a floor, per-property encumbrance lookups, broker partnerships, user-reported deals labelled self-reported
Portal listing feeds	Terms of service	Official partner or API access, own listings, a small manually collected sample with source and date recorded
Verified rents	Lease deeds not accessible	Asking rents with a stated haircut, rentals from AlphaTerra managed properties
Flood and floor-level risk	Patchy official coverage	Elevation and lake-buffer proximity as an estimated proxy
Litigation linkage	Name matching is error-prone	Flag needs legal verification, never state that a property is litigated

Canonical data model

The model has three layers: an immutable raw layer, standardised canonical entities, and append-only observations.

Entities, keys and relationships

Entity	Primary key	Foreign keys	Purpose
data_sources	source_id	none	Each source with its reliability class and licence note
raw.source_records	raw_id	source_id	Untouched original record (JSON) with content hash

Entity	Primary key	Foreign keys	Purpose
localities	locality_id	none	Standard locality names, with an alias table
developers	developer_id	none	Standard developer names and corporate ID, with an alias table
projects	project_id	developer_id, locality_id	Project with RERA ID and dates
properties	property_id	project_id, locality_id	The physical asset
listings	listing_id	property_id, source_id, latest_raw_id	One advert of a property on one source
price_observations	observation_id	listing_id, raw_id	Append-only price history
transaction_records	transaction_id	property_id, source_id	Registered transactions
rental_observations	rental_obs_id	property_id, source_id	Asking and verified rents
regulatory_records	regulatory_id	project_id, source_id	RERA, permissions, complaints, cases
users	user_id	none	Pseudonymous platform users
user_events	event_id	user_id, property_id	Funnel events
investment_transactions	investment_id	user_id, property_id	Investments, with idempotency key
model_predictions	prediction_id	none (entity type and id)	Versioned scores with uncertainty

Relationships in short: a developer has many projects. A project has many properties. A property has many listings (one per source advert). A listing has many price observations. A user has many events and many investments. Every canonical record points back to its raw record.

How the model meets the requirements

Requirement	Mechanism
Property versus listing	properties is the asset. listings is one advert, linked by property_id. Many listings can point to one property
Current versus historical price	listings holds only the current asking price. price_observations keeps every price with observed_at and ingested_at
Original versus standardised record	raw.source_records stores the original JSON and hash. Canonical tables reference it through raw_id
No duplicate properties	Unique property_fingerprint, unique rera_id on projects, and an entity_matches audit table with a review flag
No duplicate listings	Unique (source_id, source_listing_id)
No double-counted investments	Unique idempotency_key and unique payment_ref
No overwritten history	Triggers block UPDATE and DELETE on raw records and price observations
Duplicate events	event_id is the primary key and is client-generated

Key table definitions (excerpt)

The full schema is in alphaterra_schema.sql. It was written for PostgreSQL 14 or newer and should be tested against a live database before submission.

```
CREATE TABLE raw.source_records (
    raw_id BIGSERIAL PRIMARY KEY,
    source_id INT NOT NULL REFERENCES core.data_sources(source_id),
    source_record_key TEXT NOT NULL,
    payload JSONB NOT NULL,
```

```
    content_hash TEXT NOT NULL,
    fetched_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    ingestion_run_id TEXT NOT NULL,
    UNIQUE (source_id, source_record_key, content_hash)
);

CREATE TABLE core.properties (
    property_id BIGSERIAL PRIMARY KEY,
    project_id INT REFERENCES core.projects(project_id),
    locality_id INT REFERENCES core.localities(locality_id),
    bhk NUMERIC(2,1),
    carpet_area_sqft NUMERIC(10,2) CHECK (carpet_area_sqft > 0),
    builtup_area_sqft NUMERIC(10,2) CHECK (builtup_area_sqft > 0),
    floor_no INT,
    property_fingerprint TEXT NOT NULL UNIQUE,
    CHECK (carpet_area_sqft IS NULL OR builtup_area_sqft IS NULL
           OR carpet_area_sqft <= builtup_area_sqft)
);

CREATE TABLE core.listings (
    listing_id BIGSERIAL PRIMARY KEY,
    property_id BIGINT REFERENCES core.properties(property_id),
    source_id INT NOT NULL REFERENCES core.data_sources(source_id),
    source_listing_id TEXT NOT NULL,
    latest_raw_id BIGINT REFERENCES raw.source_records(raw_id),
    listing_status TEXT NOT NULL DEFAULT 'active',
    current_asking_price NUMERIC(14,2) CHECK (current_asking_price > 0),
    first_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (source_id, source_listing_id)
);

CREATE TABLE core.price_observations (
    observation_id BIGSERIAL PRIMARY KEY,
    listing_id BIGINT NOT NULL REFERENCES core.listings(listing_id),
    price_type TEXT NOT NULL DEFAULT 'asking'
        CHECK (price_type IN ('asking','registered','estimated')),
    price NUMERIC(14,2) NOT NULL CHECK (price > 0),
    observed_at TIMESTAMPTZ NOT NULL,
    ingested_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    raw_id BIGINT REFERENCES raw.source_records(raw_id),
    UNIQUE (listing_id, price_type, observed_at)
);
```

```
CREATE TABLE platform.investment_transactions (
    investment_id BIGSERIAL PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES platform.users(user_id),
    property_id BIGINT NOT NULL REFERENCES core.properties(property_id),
    amount NUMERIC(14,2) NOT NULL CHECK (amount > 0),
    status TEXT NOT NULL,
    idempotency_key TEXT NOT NULL UNIQUE,
    payment_ref TEXT UNIQUE
);
```

The entity relationship diagram should be added as an image here (for example from dbdiagram.io using the table list above).

Pipeline specification in detail

Capability	Design
Ingestion	One scheduled Python job per source. Portals via partner feeds or APIs, RERA weekly, OpenStreetMap monthly. Web collection only where terms of service and robots.txt allow it, rate-limited and with an identifying user agent. Documents (PDF) are read with text extraction and OCR as a fallback, and every extracted field carries a confidence value and a human-review flag. Internal application events arrive from the app backend and are inserted with a client-generated event_id
Data quality	Schema validation on every batch. Missing-value checks per column. Duplicate detection on source keys and on the property fingerprint. Outlier detection by interquartile range within locality and BHK. Freshness checks against a per-source SLA. Reconciliation of project facts against RERA records

Capability	Design
Entity resolution	Four tiers: (1) exact identifier such as RERA ID, (2) deterministic fingerprint of project, unit, BHK, rounded area and floor, (3) fuzzy name similarity plus geographic distance (for example within 150 metres), (4) a manual review queue for uncertain matches. Locality and developer alias tables map spelling variants. Every match is written to entity_matches with method and score
Historical tracking	Raw records are append-only. A changed record becomes a new raw row. Prices are stored as observations with observed_at (when the source showed it) and ingested_at (when AlphaTerra learned it), so late-arriving data is handled correctly. Project attributes use valid_from and valid_to
Incremental processing	Each source keeps a watermark. A content hash decides whether a record changed. Upserts use unique keys. Only the affected comparables and localities are recomputed
Reliability	Retries with exponential backoff. Respect Retry-After and rate limits. Jobs are idempotent, so running twice creates no duplicates. Bad records go to a rejected-records table instead of failing the batch. If one source is down, the others continue and that source is marked stale
Observability	A run log records job, start, end, rows read, rows written and rows rejected. Alerts for failed jobs, stale sources, null-rate jumps, and shifts in distributions (for example median price per square foot moving more than 20 percent between runs). A simple dashboard shows source freshness and record counts

Capability	Design
Governance	The source register holds licence notes. Every record is traceable to its raw record. Transformation code is versioned in git. Role-based access: analysts read, engineers write, personal data is restricted. A retention schedule applies to raw payloads and events. Sensitive information is stored separately from analytics data

Technology and justification

Choice	Reason
Python scripts	Enough for ingestion, cleaning and scoring. One engineer can maintain them
PostgreSQL as database and warehouse	Handles hundreds of thousands of rows easily. Cheap. Supports JSONB for raw payloads, constraints and triggers
Files or object storage for raw files	Cheap and durable
cron to start, a workflow tool later	Add orchestration only when the number of jobs requires it
No Spark or Kafka	Built for far larger scale. They would add cost and operating burden without benefit here. Revisit if data grows by orders of magnitude

Assumed scale: one city, tens of thousands to a few hundred thousand listings. This should be confirmed with AlphaTerra.

Intelligence products

Product A: Property Intelligence Score

Question answered: Is this property an attractive investment compared with similar properties, based on the evidence available?

Output: a score from 0 to 100, a likely range, a confidence level (low, medium, high), and a short list of reasons.

Dimension	Weight	How it is calculated
Value versus comparables	40%	Price per sq ft compared with the median of similar properties (same locality, same BHK, similar size). Score = 50 + 2 x (percent below the median), limited to 0 to 100
Rental yield	25%	Annual rent divided by price. A 2 percent yield maps to 0 and a 5 percent yield maps to 100
Location	20%	Distance to metro, employment centres, schools and hospitals, scaled against all localities
Developer and project	15%	RERA registration, delivery history, disclosed information consistency

Weights are judgement calls and are stated as assumptions. They can be tuned.

Worked example. A 2 BHK in Madhapur, 1,100 sq ft, asking 1.1 crore rupees. Price per sq ft is 10,000 and the median of 14 comparable flats is 11,000, so it is about 9 percent cheaper: value score 68. Rent of 38,000 per month gives a 4.15 percent yield: yield score 72. Location 80. Developer 60. Final score = 0.40 x 68 + 0.25 x 72 + 0.20 x 80 + 0.15 x 60 = 70.

Missing data. A missing dimension is removed and its weight is spread across the others. It is never filled with a guess and never scored as zero. Confidence falls with missing dimensions, few comparables, and reliance on asking prices instead of verified values.

Uncertainty. The score is shown with a range and a confidence level, for example: 70 (likely range 62 to 77), confidence medium, based on 14 comparable listings, rent is an asking rent.

Risk flags from Product C are shown next to the score and are never hidden inside the average.

Score versus prediction. The score compares today's evidence with similar properties. It does not predict future returns, because that needs transaction history AlphaTerra does not have. Every score carries this statement: this score compares current evidence with similar properties. It is not a prediction of returns and not investment advice. A high score never means a guaranteed good investment.

Limitations. Relies on asking prices. Weights are judgement. Few comparables lowers confidence. Cannot see legal disputes unless Product C flags them.

Decisions supported. Which properties to feature, which to list for investors first, which need due diligence before listing.

Product B: Locality Intelligence

Question answered: Which Hyderabad localities are becoming more or less attractive relative to comparable localities?

Indicators (monthly, per locality):

Indicator	Definition
Mix-adjusted price per sq ft	Median price per sq ft per segment (BHK and size band), combined with a fixed reference mix so a change in property mix does not look like a price change
Transaction activity	Count of registered transactions, where available
Rental yield	Median asking rent divided by median price, with a stated haircut on rent
Listing inventory	Number of active listings
Time on market	Median days from first seen to sold or withdrawn
Price-cut share	Share of active listings with a price reduction in the last 60 days
Accessibility	Distance and travel time to metro and employment centres
Platform demand	Detail views, enquiries and saves per active listing on AlphaTerra

Comparing localities with different property mixes. Prices are compared inside segments, then combined with a fixed reference mix (the same weights for every locality and every month). This is a stratified, like-for-like index. A locality with many small flats is not treated as cheaper than one with many large flats.

Unequal data coverage. A locality needs a minimum number of listings per month (for example 30) to be scored. Below that it shows insufficient evidence. Values are shrunk towards the city or peer-group average when samples are small. Bootstrap intervals show the uncertainty of each index.

Trend labels. A locality's index is compared with its peer group. A locality is labelled strengthening or weakening only if the change is larger than the noise band and at least two independent indicators agree (for example price index and falling time on market, or price index and rising platform demand). Otherwise it is labelled stable or insufficient evidence.

Rule: a locality is never labelled appreciating only because the average asking price rose. Reasons: the property mix may have changed, asking prices are not transaction prices, a few listings can move an average, and a change relative to peers matters more than an absolute change.

Decisions supported. Where to source more inventory, which localities to feature, where to focus marketing, where to request due diligence.

Limitations. Needs several months of history before trends mean anything. Transaction data is sparse. Platform demand reflects AlphaTerra's own audience and not the whole market.

Product C: Property and Listing Risk Intelligence

Question answered: Which listings or records contain inconsistencies or potential risks that need checking?

Two separate things. An automated anomaly flag is a statistical or rule-based signal. It is not evidence of wrongdoing. A verified finding is a flag that a person has checked against documents. The system never uses words such as fraud or illegal for a flag. It says inconsistency requiring verification.

Alert	Example	Automated?	Next step
Duplicate listing	Same fingerprint on several sources with different prices	Yes	Auto-link, flag price conflict

Alert	Example	Automated?	Next step
Inconsistent area	Carpet area larger than built-up area, or area differs more than 10 percent across sources	Yes	Flag, human review if large
Price outlier	Price per sq ft far from comparables (interquartile or z-score rule)	Yes	Flag, human review if extreme
Unexplained price change	Change above 15 percent within a week, no status change	Yes	Flag, human review
Conflicting project information	Developer on listing differs from RERA promoter, or possession dates differ	Rule yes, judgement human	Human review
Missing regulatory information	Under-construction project with no RERA ID	Yes	Flag as needs verification
Document versus listing mismatch	Sanctioned floors differ from claimed floors	Extraction automated, judgement human	Human or legal review
Stale listing	Not updated for more than 30 days	Yes	Auto-mark stale
Developer delay pattern	Repeated late completions in RERA history	Indicator automated, interpretation human	Show as context, human review

Workflow. Detected, under review, then one of: confirmed issue, cleared, or unresolved. Only confirmed issues are shown to investors as verified findings, with the evidence and the date.

Severity. Low (data quality), medium (needs checking before listing), high (do not list until reviewed).

Measuring false positives. Reviewers record an outcome for every alert. Precision per rule = confirmed alerts divided by confirmed plus cleared alerts. Track precision, review rate and time to resolve for each rule. Thresholds are tuned when precision is low. A

random sample of unflagged listings is audited regularly to estimate missed problems.

Legal care. Alerts are internal until reviewed. Wording avoids accusation. Litigation and encumbrance items always go to human or legal review. This is not legal advice and should be reviewed by counsel.

Decisions supported. Which listings to publish, hold or correct, and which developers need closer review.

Investor behaviour intelligence

Purpose: use AlphaTerra's own application data to understand the journey from property discovery to funded investment.

Events required

Event	Key properties
impression	user_id, property_id, position, channel
detail_view	user_id, property_id, session_id
saved_listing	user_id, property_id
enquiry	user_id, property_id
kyc_initiated, kyc_completed	user_id, step, error code if any
investment_initiated	user_id, property_id, amount
payment_confirmed	user_id, payment_ref
investment_funded	user_id, investment_id
investment_cancelled, refund	user_id, investment_id, reason

Every event carries event_id (unique), pseudonymous user_id, timestamp, device and acquisition channel or campaign.

Measurement

- **Stage conversion:** users who reach a stage divided by users who reached the previous stage, within a fixed window (for example 30 days from first detail view), by cohort.

- **Friction:** the stages with the biggest drop-off, broken down by device, channel, property type and KYC step or error code. Time between stages shows delays.
- **Channels:** compare acquisition channels by funded investments per 100 sign-ups and by cost per funded investor, not by clicks or page views.
- **Properties with meaningful interest:** enquiries, saves and investment initiations per 100 detail views, with a minimum number of views before a property is ranked.

Investors versus browsers

A propensity model estimates the chance that a user funds an investment within 30 days. Start with a transparent rule-based score, then a logistic regression, and move to a more complex model only if it clearly improves results.

Features from the first days of activity: number of distinct properties viewed, repeat views of the same property, time on detail pages, saves, enquiry made, KYC started, consistency of price range, number of sessions. Label: funded within 30 days. Features must not use information that arrives after the prediction time (leakage). Evaluation uses a time-based split, with AUC, calibration and precision at the top k users. The output is a ranking for prioritising follow-up, not a claim about any individual.

Correlation and causation

Users who complete KYC invest more, but KYC completion is a consequence of intent. It does not show that pushing people through KYC creates investments. The same applies to users who view many pages or save many listings. Causal claims are made only from experiments.

Proposed experiment: recommendations

Item	Design
Question	Does the new property recommendation approach increase funded investments, not just page views?
Unit and assignment	Users, randomly split 50/50 between current and new recommendations, fixed for the whole test
Primary metric	Funded investments per exposed user within 30 days of first exposure

Item	Design
Secondary metrics	Enquiry rate, investment initiation rate, time to first enquiry
Guardrails	Cancellations, refunds, support tickets, share of recommended listings with risk flags. Page views are tracked but never used as the success metric
Sample size (illustration)	Baseline 2 percent funded conversion, detecting a lift to 2.4 percent (20 percent relative), 80 percent power, 5 percent significance: about 21,000 users per arm. This is an assumption to be recalculated with the real baseline
Duration	At least two full weeks plus the 30-day outcome window
Discipline	Metrics fixed in advance, check for sample ratio mismatch, no early stopping on interim results
If traffic is too low	Run longer, or test a higher-volume proxy such as investment initiation, and state clearly that it is a proxy

Privacy, access and retention

This follows the principles of India's Digital Personal Data Protection Act 2023 and should be confirmed with legal counsel.

- Consent and purpose: collect behavioural data only for stated purposes, with recorded consent.
- Minimisation: pseudonymous user_id in analytics. Names, phone numbers, KYC documents and payment details stay in separate restricted systems.
- Access: role-based. Analysts see pseudonymised data. Reports show aggregates only, with a minimum group size.
- Retention: raw events kept for a defined period (for example 24 months, an assumption), then aggregated. KYC and payment records follow regulatory retention rules.
- Rights: support deletion and access requests. Keep audit logs of who accessed what.

Prioritisation

There is capacity to start one major implementation. The four options are compared on a 1 to 5 scale where 5 is best (so for cost and risk, 5 means cheap or low risk). The scores are my judgement based on the assumptions below.

Option	Business impact	Implementation cost	Data availability	Operational risk	Time to first result
A. Property Intelligence Score	5	2	3	3	2
B. Locality Intelligence	3	2	2	3	1
C. Listing Risk Intelligence	4	4	4	3	5
D. Investor behaviour system	4	4	3	4	4

Option	Reasoning	Estimated time to first measurable result
A	Core value for investors, but needs comparables, rents and several inputs. A misleading score creates reputational risk	6 to 8 weeks
B	Useful but needs months of history before trends are meaningful, and transaction data is thin	3 to 4 months
C	Works on listings and RERA data that can be obtained now. Builds the matching and cleaning that A and B need	3 to 4 weeks
D	Uses internal data, no external legal issues, but depends on events being instrumented. Funnel report is fast, experiments take longer	2 to 4 weeks for the funnel, 6 to 8 weeks for an experiment

Weights used: impact 30 percent, data availability 25 percent, cost 15 percent, risk 15 percent, time to result 15 percent. Weighted totals: C 4.00, D 3.75, A 3.30, B 2.30.

Recommendation

Build Product C, Property and Listing Risk Intelligence, first, scoped as a data trust layer.

Why: (1) its data can be obtained lawfully now, (2) it creates the duplicate detection, entity resolution and consistency checks that Products A and B depend on, (3) for a platform that takes investor money, trust is a direct business risk, (4) it gives a measurable result in about four weeks, (5) it carries little modelling risk.

Assumptions: the platform lists properties from external sources, only one major build is possible, listing data quality is currently unmeasured, and funnel events may not yet be reliably tracked.

Evidence that would change this recommendation:

- If funnel events are already reliable and show a large drop (for example at KYC or between enquiry and investment), Option D should come first. With data availability scored 5 instead of 3, D scores 4.25 and overtakes C.
- If an initial audit shows listing data is already clean (very few inconsistencies), the value of C is lower and A becomes the better first build.
- If lawful access to listing or RERA data cannot be obtained, C loses its data advantage and D should come first.

Business case

First feature: a listing risk and data trust layer. Minimum scope for about four weeks: ingest two or three listing sources plus RERA data, store raw and canonical records, match duplicates, run the first six rules (duplicates, area inconsistency, price outlier, price change, missing RERA, stale listing), a review queue for flagged items, and a weekly quality report.

Expected value:

- Fewer misleading or duplicated listings reaching investors, which supports trust and conversion.
- Fewer cancellations and refunds linked to listing problems.
- Less analyst time spent on manual checking.
- Clean, matched data that makes Products A and B possible.

Principal risks:

Risk	Mitigation
Too many false alerts waste reviewer time	Measure precision per rule, tune thresholds, start with high-precision rules
Legal exposure from wording	Neutral language, human review before any external statement, counsel review
Data access or terms-of-service limits	Partner feeds, permitted sources, documented licence notes
Thin coverage of RERA or documents	Report coverage, mark gaps as unavailable
Review capacity too small	Severity levels, review only high and medium first

How success is measured (targets are proposals to be set after a baseline):

Metric	Definition
Duplicate rate	Share of active listings that duplicate another listing
Data completeness	Share of listings with all core fields filled
RERA match rate	Share of under-construction listings matched to a RERA record
Alert precision	Confirmed alerts divided by confirmed plus cleared, per rule. For example 70 percent or higher for high-severity rules
Time to resolve	Median time from alert to decision
Downstream effect	Cancellation or refund rate and enquiry-to-investment conversion for reviewed versus unreviewed listings, compared carefully because this is observational

Proof of concept

The proof of concept is a small pipeline on synthetic data for Madhapur, Khajaguda, Nanakramguda and Narsingi. It was run end to end.

Script	Purpose	Output
make_data.py	Creates 200 synthetic listings with a fixed seed	data/properties.csv
validate.py	Injects four deliberate errors, detects them, cleans the data	data/properties_clean.csv
cluster2.py	KMeans with 3 clusters on price per sq ft and total price, then a locality summary	data/properties_segmented.csv, data/locality_summary.csv
charts.py	Draws three charts	charts/*.png
run_all.py	Runs all steps in order	all of the above

How to run: install Python 3.9 or newer, run pip install -r requirements.txt, then python run_all.py.

Validation results. Out of 201 rows (200 plus one injected duplicate): 1 missing area, 1 duplicate, 1 impossible area, 1 impossible price per sq ft. After cleaning, 197 rows remain.

Clustering results (synthetic data, values approximate).

Segment	Average total price	Price per sq ft	Typical flat
Low budget	about 98 lakh	about 7,300	2 BHK, about 1,350 sq ft
Mid range	about 1.13 crore	about 10,100	2 BHK, about 1,100 sq ft
High	about 2.0 crore	about 9,200	4 BHK, about 2,200 sq ft

Locality	Median price per sq ft	Share low budget
Narsingi	7,684	67%
Khajaguda	7,964	60%
Nanakramguda	9,146	34%
Madhapur	10,063	16%

Reading the result. Narsingi and Khajaguda come out as the budget-friendly localities and Madhapur as the premium one. A first version that clustered on five features, including flat size, labelled Madhapur mostly low budget because the groups were driven by size. Clustering on price per sq ft and total price fixed this.

Honest limits. The data is synthetic and was built with Madhapur priced higher and Narsingi lower, so the clustering recovers what was put in. It proves the method, not real Hyderabad facts. Floor numbers are random, so differences in high-rise share between localities are noise and no high-rise conclusion is claimed. With real data, high-rise would use the building's total floors.

Assumptions and limitations

- Scale: one city, tens of thousands to a few hundred thousand listings.
- Proof-of-concept data is synthetic. Base prices per locality are invented.
- Valid ranges used in validation: area 300 to 5,000 sq ft, price 3,000 to 25,000 per sq ft.
- Three clusters were chosen because the brief asks for low, mid and high. Cut-offs come from the clustering, not from fixed price limits.
- Score weights, alert thresholds, the 30-day outcome window, retention periods and the success targets are proposals to be calibrated.
- Asking prices are not transaction prices. Rent figures are asking rents.
- Legal and privacy statements are general and are not legal advice. Portal terms, RERA portal terms and the DPDP Act should be checked by counsel before collection or launch.
- The SQL was written for PostgreSQL 14 or newer and should be tested on a live database.

Appendix A: Proof-of-concept code

Single quotes are used below. The files in the project folder are equivalent.

make_data.py

```
import numpy as np
import pandas as pd
import os

np.random.seed(42)
```

```
os.makedirs('data', exist_ok=True)

base_price = {'Madhapur': 10000, 'Khajaguda': 8000,
              'Nanakramguda': 9000, 'Narsingi': 7500}
avg_area = {1: 650, 2: 1100, 3: 1600, 4: 2400}

rows = []
for i in range(200):
    loc = np.random.choice(list(base_price))
    bhk = int(np.random.choice([1, 2, 3, 4], p=[0.1, 0.4, 0.35, 0.15]))
    area = int(avg_area[bhk] * np.random.normal(1, 0.1))
    pps = int(base_price[loc] * np.random.normal(1, 0.15))
    rows.append({'listing_id': i + 1, 'locality': loc, 'bhk': bhk,
                 'area_sqft': area, 'floor': int(np.random.randint(1, 31)),
                 'price_per_sqft': pps, 'asking_price': area * pps})

df = pd.DataFrame(rows)
df.to_csv('data/properties.csv', index=False)
```

validate.py

```
import pandas as pd

df = pd.read_csv('data/properties.csv')

# Inject deliberate errors to simulate messy real data
df.loc[5, 'area_sqft'] = None
df.loc[10, 'area_sqft'] = 50000
df.loc[15, 'price_per_sqft'] = 90000
df = pd.concat([df, df.iloc[[20]]]).reset_index(drop=True)

print('Total rows:', len(df))
print(df.isnull().sum())
print('Duplicates:', df.duplicated(subset=['listing_id']).sum())

clean = df.drop_duplicates(subset=['listing_id'])
clean = clean.dropna(subset=['area_sqft'])
clean = clean[clean['area_sqft'].between(300, 5000)]
clean = clean[clean['price_per_sqft'].between(3000, 25000)]
clean.to_csv('data/properties_clean.csv', index=False)
print('Rows after cleaning:', len(clean))
```

cluster2.py

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv('data/properties_clean.csv')
X = StandardScaler().fit_transform(df[['price_per_sqft', 'asking_price']])

df['cluster'] = KMeans(n_clusters=3, random_state=42,
n_init=10).fit_predict(X)
order = df.groupby('cluster')['asking_price'].mean().sort_values().index
df['segment'] = df['cluster'].map(dict(zip(order, ['Low budget', 'Mid range',
'High'])))

df['high_rise'] = df['floor'] >= 20
df['low_budget'] = df['segment'] == 'Low budget'
summary = df.groupby('locality').agg(
    properties=('listing_id', 'count'),
    median_price_per_sqft=('price_per_sqft', 'median'),
    median_price=('asking_price', 'median'),
    pct_low_budget=('low_budget', 'mean'),
    pct_high_rise=('high_rise', 'mean'),
).round(2)
print(summary)

df.to_csv('data/properties_segmented.csv', index=False)
summary.to_csv('data/locality_summary.csv')
```

charts.py draws a scatter of segments, a bar chart of median price per sq ft by locality, and a stacked bar of segment mix by locality, saved to the charts folder using matplotlib.

run_all.py

```
import subprocess
import sys

steps = ['make_data.py', 'validate.py', 'cluster2.py', 'charts.py']
for script in steps:
    print('Running', script)
    result = subprocess.run([sys.executable, script])
    if result.returncode != 0:
        print('STOPPED:', script, 'failed')
        sys.exit(1)
print('All steps finished.')
```

requirements.txt: pandas, numpy, scikit-learn, matplotlib.

Appendix B: Data dictionary (properties.csv)

Column	Meaning	Unit or range
listing_id	Unique number for each listing	integer
locality	Madhapur, Khajaguda, Nanakramguda or Narsingi	text
bhk	Number of bedrooms	1 to 4
area_sqft	Area	square feet
floor	Floor number	1 to 30
price_per_sqft	Asking price per square foot	rupees
asking_price	area_sqft multiplied by price_per_sqft	rupees
cluster	KMeans group number (added by cluster2.py)	0 to 2
segment	Low budget, Mid range or High (added by cluster2.py)	text